Version 0.3.0 _ LLM Planner + Python Executor

At this stage of development, the implemented system represents a **lightweight, controlled, and semantically aware LLM-based co-pilot** for automating the Code_Aster simulation workflow. Although conceptually related to a LangChain Agent, the current architecture deliberately avoids using LangChain's built-in agent framework or LangGraph dependency, focusing instead on stability, interpretability, and computational efficiency.

The system integrates a locally hosted language model (via Ollama, such as TinyLlama or Mistral) directly with a Python-based pipeline that contains all Code_Aster tools—geometry generation, meshing, solver file creation, export file generation, execution, and visualization. In this design, the large language model is not an autonomous agent capable of invoking functions. Rather, it acts as a **planner** that produces a structured natural-language plan describing the logical steps required to simulate a beam model. This plan is then processed by a **semantic controller** written in Python, which interprets the textual output, detects key terms, and determines which tools to execute.

Unlike a LangChain or LangGraph Agent, which embeds reasoning loops and function-calling mechanisms within a managed state graph, this co-pilot architecture externalizes the reasoning logic: the LLM provides context and guidance, while deterministic Python code governs the execution. This separation ensures full transparency and deterministic control of the computational workflow. The system analyzes the LLM's plan, identifies semantic cues such as "geometry," "mesh," "solver input," "export," "run," and "visualization," and then triggers the corresponding functions from the modular `tools` library. If the LLM omits a step, the controller automatically detects missing output files and executes the appropriate functions to complete the pipeline.

This approach yields several methodological advantages. It avoids the overhead and versioning complexity of modern LangChain Agent APIs while retaining interpretability, modularity, and reproducibility—key criteria in scientific and engineering applications. Furthermore, the absence of hidden reasoning graphs and external dependencies makes the framework computationally lightweight, suitable for local execution on a standard laptop without GPU acceleration.

In essence, the implemented co-pilot functions as an **intelligent orchestration layer** between human-interpretable planning and deterministic numerical execution. It blends the descriptive reasoning of a language model with the reliability of explicit engineering computation, forming an intermediate yet robust step toward a full agent-driven Code_Aster automation system.